does expire. If it does, then use the MacaroonsBuilder.modify() method to add a new first-party caveat to the macaroon with the preceding syntax. `The java.time.Instant` class already returns the timestamp in the correct format. Update the create method to match the listing.

Listing 9.16 Adding an expiry caveat

```
@Override
public String create(Request request, Token token) {
var identifier = delegate.create(request, token);
var macaroon = MacaroonsBuilder.create("",
macKey.getEncoded(), identifier);
if (token.expiry != Instant.MAX) { #A
macaroon = MacaroonsBuilder.modify(macaroon) #A
.add_first_party_caveat("time < " + token.expiry) #A
.getMacaroon(); #A
}
return macaroon.serialize();
}
```

#A If the token expires then add a first-party caveat enforcing the expiry time.

To satisfy the caveat, you need to add the `TimestampCaveatVerifier` to the `MacaroonsVerifier` in the `read` method of the token store. The macaroons library will try to match each caveat to a verifier that is able to satisfy it. In this case, the verifier checks that the current time is before the expiry time specified in the caveat. If the verification fails, or if the library is not able to find a verifier that matches a caveat, then the macaroon is rejected. This means that the API must explicitly register verifiers for all